

PROJECT CHARTER

Autonomous AI Customer Support & Resolution Agent

Thirilose Jones Nithish R · v2 — AI-powered customer support system using customer-support conversations, e-commerce data, and a policy knowledge base

1. Business Problem

E-commerce customer support teams handle a large number of repetitive queries related to order status, delivery, cancellations, refunds, payments, returns, and general policies. Customers often need information that is spread across order records, customer records, and company policies.

A traditional chatbot can generate responses, but it may not have access to the customer's actual order information and can provide inaccurate answers when business rules are involved.

The problem is to build a support system that can understand customer queries, retrieve relevant information, and provide reliable responses using actual business data and company policies.

2. My Objective

Build an AI-powered customer support agent that can:

- Understand and classify different types of customer queries
- Answer common customer-support questions using a knowledge base
- Retrieve customer, order, product, and payment information when required
- Provide context-aware responses based on the available information
- Handle simple support requests automatically
- Identify queries that require human assistance and recommend escalation
- Maintain conversation context during a support session

The primary objective is to demonstrate how Generative AI can be combined with structured business data and RAG to automate routine customer support.

3. Dataset — Customer Support + E-Commerce Data

The project uses two main datasets and one custom knowledge base.

[Drive Link for both the datasets](#)

Dataset 1 — Bibtex Customer Support Dataset

File: Bibtex_Sample_Customer_Support_Training_Dataset_27K_responses-v11

This dataset contains customer-support queries, categories, intents, and example responses.

Column	Role
instruction	Customer's query/request
category	Broad support category
intent	Specific customer intent
response	Example/reference response

Purpose: Used to understand the types of customer problems the AI agent needs to handle and to evaluate intent recognition and response generation.

Dataset 2 — Olist Brazilian E-Commerce Dataset

The e-commerce data is divided into related tables:

File	Role	Important Columns
customers_dataset	Customer information	customer_id, customer_unique_id, customer_city, customer_state
orders_dataset	Order lifecycle	order_id, customer_id, order_status, purchase/approval/delivery timestamps, estimated delivery date
order_items_dataset	Products within orders	order_id, order_item_id, product_id, seller_id, price, freight_value
products_dataset	Product information	product_id, product_category_name
order_payments_dataset	Payment information	order_id, payment_type, payment_installments, payment_value

Purpose: Used as the agent's structured business data source for answering customer-specific questions.

Dataset 3 — Custom Company Knowledge Base

I will create a small fictional company knowledge base containing:

- Return Policy
- Refund Policy
- Cancellation Policy
- Shipping & Delivery Policy
- Warranty Policy
- Frequently Asked Questions

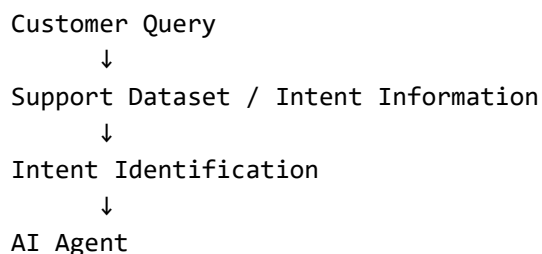
These documents will be converted into a searchable knowledge base for the RAG component.

Note: This is not a downloaded dataset. It is created specifically for the project so that the AI has clear and controlled company policies to reference.

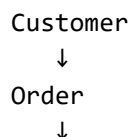
4. How the Data Relates

The project has two different data flows.

Customer Support Flow



Business Information Flow



Order Items



Products



Payments

The customer-support dataset helps define what the customer is asking, while the e-commerce tables provide the actual business information required to answer the query.

The policy documents provide the rules required for policy-related questions.

5. My Chosen Problem Lens

Autonomous Customer Support

Primary question I'm answering:

Can an AI agent automatically understand common customer-support queries, retrieve the required business or policy information, and provide an appropriate response without requiring a support representative for every routine request?

The project focuses on common e-commerce support scenarios such as:

- Order tracking
- Delivery status
- Cancellation
- Return eligibility
- Refund-related questions
- Payment-related questions
- Product information
- Shipping policies
- General FAQs

The project is intentionally focused on simple and practical autonomous support, rather than building a fully autonomous enterprise support platform.

6. Core AI Solution

The proposed solution follows this updated workflow, aligned with the LangChain / LlamaIndex / PGVector stack:

Customer Query (Text or Voice via Whisper)



Intent Understanding (LLM / LangChain)



Knowledge Ingestion → Vector Embeddings Store (LlamaIndex + PGVector)



RAG Retrieval



LLM Response with Guardrails



Human Fallback + Auto Ticket Creation (if needed)

For structured business data lookups (orders, payments, products), the agent queries PostgreSQL directly instead of the vector store, while policy/FAQ questions are routed through the RAG pipeline (LlamaIndex + PGVector).

For example:

Customer:

“Where is my order?”

Intent → Order Status



Search Order Database (PostgreSQL)



Retrieve Order Status



Generate Response

Customer:

“Can I return my product after 10 days?”

Intent → Return Policy



Search Knowledge Base (RAG – LlamaIndex + PGVector)



Retrieve Return Policy



Generate Response

7. Key Features

The first version of the system will include:

- AI Customer Chat Interface (text and voice input)
- Customer Query/Intent Understanding
- RAG-based Policy & FAQ Search
- Order Information Lookup
- Product Information Lookup
- Payment Information Lookup
- Context-aware responses
- Basic conversation memory
- Human escalation for unsupported or uncertain queries

The system will not directly perform sensitive financial actions such as issuing refunds. It will provide the relevant information and recommend escalation where necessary.

8. Main Scenarios to Demonstrate

Scenario	Data/AI Component
“Where is my order?”	Orders database
“Is my order delivered?”	Orders database
“What product did I purchase?”	Orders + Products
“How much did I pay?”	Orders + Payments

Scenario	Data/AI Component
“What payment method did I use?”	Payments
“Can I return my product?”	Orders + RAG
“What is your refund policy?”	RAG
“Can I cancel my order?”	Orders + RAG
“I have a problem that you cannot resolve.”	Human escalation

9. Tech Stack

Updated to match the finalized architecture (Python, LangChain, LlamaIndex, PGVector, FastAPI, Whisper), with PostgreSQL retained for structured business data:

Tool	What I'm using it for
Python	Core application and AI logic
LangChain	Agent orchestration, RAG pipeline, and guardrails
LlamaIndex	Document ingestion and indexing for the knowledge base
PGVector	Vector embeddings storage and similarity search
PostgreSQL	Structured data — customers, orders, products, and payments
FastAPI	Backend API
Whisper	Voice-to-text input for the chat interface
Web UI (Chat Interface)	Customer-facing chat interface supporting text and voice input
Git & GitHub	Version control and final submission

10. Final Outcome / Deliverable Type

The final deliverable is a WEB APPLICATION with a chat interface that supports both TEXT and VOICE input (via Whisper). The frontend chat UI communicates with a FastAPI backend, which routes each customer query to: (1) RAG retrieval (LlamaIndex + PGVector) for policy/FAQ questions, (2) PostgreSQL for structured business data lookups (orders, products, payments), or (3) the LLM directly for simple conversational responses. Guardrails validate responses before they reach the customer, and unresolved or uncertain queries trigger human fallback with automatic support-ticket creation.

11. Basic Agent Workflow

The agent will follow a simple decision process:

```

Receive Customer Query (Text or Voice)
↓
Understand Intent
↓
Identify Required Information
↓
Retrieve Data / Policy (PostgreSQL or RAG)
↓
Generate Response (with Guardrails)
↓
Check Whether Query Can Be Resolved
↓

```

Yes → Answer Customer

No → Human Fallback + Auto Ticket Creation

The goal is not to make the agent perform complex autonomous actions. The focus is on reliable information retrieval and response generation.

12. Evaluation Criteria

The system will be evaluated based on:

- Intent identification accuracy
- Correct retrieval of order/customer information
- Relevance of retrieved policy information
- Accuracy of generated responses
- Ability to maintain conversation context
- Appropriate handling of unsupported queries
- Appropriate human-escalation decisions

Example evaluation:

Customer Query



Expected Intent



Agent Intent



Retrieved Information



Generated Response



Correct / Incorrect

13. Deliverables Checklist

Folder	Contents
01 Data Preparation	Data cleaning scripts, processed datasets, data dictionary
02 Database	PostgreSQL schema, tables, sample queries
03 Knowledge Base	Return, refund, shipping, cancellation and FAQ documents
04_RAG	Document processing (LlamaIndex), embeddings and retrieval implementation (PGVector)
05 AI Agent	Agent logic (LangChain), intent handling and response generation
06 Web App	FastAPI backend + Web chat UI with text and voice (Whisper) input
07 Evaluation	Test queries, evaluation results and validation notes
08 Documentation	Project report, architecture, screenshots and methodology
09 Presentation	Final project presentation
README.md	Project overview, setup instructions and usage

14. Final Project Outcome

The final application will provide a web-based AI customer-support interface with a chat window where a customer can type or speak (via Whisper) questions in natural language.

The AI agent will determine whether the query requires:

- Knowledge → RAG (LlamaIndex + PGVector)
- Business information → PostgreSQL Database
- Simple response → LLM
- Unresolved/uncertain issue → Human escalation with auto-ticket creation

The final goal is to demonstrate a practical application of Generative AI, RAG, structured business data, and simple agent-based decision making to improve the efficiency of e-commerce customer support.